# Wrapping Condition Number
# Iterative Methods in Linear Algebra

Alexandra A. Gomes

March 1, 2026

# Last Lecture: Condition Number

- The **condition number** of a non-singular matrix $\mathbf{A}$ with respect to a norm $\|\cdot\|$ is defined as

$$\kappa(\mathbf{A}) = \|\mathbf{A}\| \left\|\mathbf{A}^{-1}\right\|.$$

- Note that $\kappa(\mathbf{A})$ depends on the norm: $\kappa_2(\mathbf{A})$ or $\kappa_\infty(\mathbf{A})$, for condition numbers corresponding to the $\ell_2$-norm or $\ell_\infty$-norm.
- relative error is also called forward error

$$\frac{\|\mathbf{x} - \tilde{\mathbf{x}}\|}{\|\mathbf{x}\|} \leq \kappa(\mathbf{A}) \frac{\|\tilde{\mathbf{r}}\|}{\|\mathbf{b}\|}$$

that is, the residual is magnified by the condition number of the matrix.

# Backward Error Analysis: Another Lens

- We never get the exact solution $x$; we get a computed $\tilde{x}$. The question is: **did we solve a problem close to the one we intended?**

$Ax = b$

- **Backward error** looks at perturbations $(\delta\mathbf{A}, \delta\mathbf{b})$ such that

$$\boxed{(\mathbf{A} + \delta\mathbf{A})\,\tilde{\mathbf{x}} = \mathbf{b} + \delta\mathbf{b}.}$$

$\delta A\tilde{x} - \delta b = \underbrace{b - A\tilde{x}}_{=\tilde{r}}$

- The computed solution $\tilde{\mathbf{x}}$ may be viewed as the exact solution of the perturbed system.
- Substituting $\tilde{\mathbf{r}} = \mathbf{b} - \mathbf{A}\tilde{\mathbf{x}}$ yields

$$\boxed{\tilde{\mathbf{r}} = \delta\mathbf{A}\tilde{\mathbf{x}} - \delta\mathbf{b},}$$

that is, the residual can be explained by small perturbations in $\mathbf{A}$ and $\mathbf{b}$.

- A small residual means $\tilde{\mathbf{x}}$ is the exact solution of a nearby system.

# Backward Error Analysis: Another Lens

$0 \le x \le 1$

abot of arithmetic + bounding

- Then (...), the forward error is bound above as

$$\frac{\|\mathbf{x} - \tilde{\mathbf{x}}\|}{\|\mathbf{x}\|} \le \frac{\kappa(\mathbf{A})}{1 - \kappa(\mathbf{A})\,\|\delta\mathbf{A}\| / \|\mathbf{A}\|} \left( \frac{\|\delta\mathbf{b}\|}{\|\mathbf{b}\|} + \frac{\|\delta\mathbf{A}\|}{\|\mathbf{A}\|} \right),$$

$A + \delta A$ $\quad$ $b + \delta b$

that is, the relative solution error is bounded by the relative perturbations in **A** and **b**;

- Stable algorithm (small $\delta\mathbf{A}$, $\delta\mathbf{b}$ yield small perturbations to the solution) on a well-conditioned problem results in a small error. That is, even a stable algorithm cannot overcome an ill-conditioned problem.

# Wrapping Up: Condition Number

- rule of thumb for loss of significance:
  - if $\kappa(\mathbf{A}) = \mathcal{O}(10^{\alpha})$, we loose $\alpha$ significant digits;
  - if the linear system is solved in $t$-digit floating-point arithmetic, only $t - \alpha$ digits in the solution can be trusted;
  - so, if the previous example was to be solved with double precision ($t$ is about 16; in single precision, $t$ is about 7), for $\alpha = 4$, the solution would have about 16-4 $= 12$ significant digits;
- do not infer about $\kappa(\mathbf{A})$ from $\det \mathbf{A}$;
- if $\mathbf{Q}^T\mathbf{Q} = \mathbf{I}$, $\kappa_2(\mathbf{Q}) = 1$;
- if $\mathbf{A}$ is SPD, with eigenvalues $\lambda_1 \geq \ldots \lambda_n > 0$, then

$$\kappa_2(\mathbf{A}) = \|\mathbf{A}\|_2 \|\mathbf{A}^{-1}\|_2 = \frac{\lambda_1}{\lambda_n}.$$

# Summary: Solution Methods for Linear Equations

Goal: to solve $\mathbf{Ax} = \mathbf{b}$, $\mathbf{A} \in \mathbb{R}^{n \times n}$.
Question: which method matches the matrix and the scale?

- dense direct methods (LU/Cholesky):
  - runtime depends on size; independent of data, structure, or sparsity;
  - work well for $n$ up to a few thousand;
- sparse direct methods:
  - runtime depends on size, sparsity pattern; (almost) independent of data;
  - can work well for $n$ up to $10^4$ or $10^5$ (or more);
  - requires good heuristic for fill-in minimizing ordering;
- iterative methods:
  - runtime depends on data, size, sparsity, required accuracy;
  - requires tuning, preconditioning, and stopping rule;
  - good choice in many cases; only choice for $n = 10^6$ or larger.

# Iterative Method

- start with an initial guess of the solution $\mathbf{x}^{(0)}$:
  - may be as simple as $\mathbf{x}^{(0)} = \mathbf{0}$;
  - we may have a good initial guess;
- at every iteration, compute $\mathbf{x}^{(k+1)}$, from $\mathbf{x}^{(k)}$;
- stop when desired accuracy is reached;
- we generated $\left\{\mathbf{x}^{(k)}\right\}_{k=0}^{\infty}$ that converges to $\mathbf{x}$.

# Types of Iterative Methods

1. stationary methods:
   - Jacobi;
   - Gauss-Seidel;
   - successive over-relaxation (SOR);
   - multigrid;
2. Krylov methods:
   - conjugate gradient;
   - gradient descent.

# Stationary Methods

$A = M - N \Rightarrow N = M - A$

$$\mathbf{Ax} = \mathbf{b} \longrightarrow (M-N)x = b \Leftrightarrow Mx = Nx + b$$

$$\Leftrightarrow \underbrace{Mx = (M-A)x + b}$$

$$\Leftrightarrow x = M^{-1}(M-A)x + M^{-1}b$$

- let $\mathbf{A} = \mathbf{M} - \mathbf{N}$ so that $\mathbf{Ax} = \mathbf{b}$ becomes

$$\begin{aligned} \mathbf{Mx} &= \mathbf{Nx} + \mathbf{b} \\ &= (\mathbf{M} - \mathbf{A})\mathbf{x} + \mathbf{b} \\ \boxed{\mathbf{x}} &\boxed{= (\mathbf{I} - \mathbf{M}^{-1}\mathbf{A})\mathbf{x} + \mathbf{M}^{-1}\mathbf{b};} \end{aligned}$$

- leads to an iteration of the form

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{M}^{-1}\left(\mathbf{b} - \mathbf{A}\mathbf{x}^{(k)}\right) = \mathbf{x}^{(k)} + \mathbf{M}^{-1}\mathbf{r}^{(k)};$$

- called *stationary* because iteration matrix does not change;
- the hard part is how to do the splitting of $\mathbf{A}$ into $\mathbf{M}$ and $\mathbf{N}$;

$$A = \begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}}_{M} - \underbrace{\begin{bmatrix} 0 & 0 \\ -2 & 0 \end{bmatrix}}_{N} \qquad \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

$$\underbrace{x^{(0)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}}; \quad k=0; \quad k+1=1 \quad \underbrace{x^{(1)}} = \underbrace{x^{(0)}}_{\begin{bmatrix} 0 \\ 0 \end{bmatrix}} + M^{-1}\left(\begin{bmatrix} 1 \\ 2 \end{bmatrix} - \begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix}\begin{bmatrix} 0 \\ 0 \end{bmatrix}\right)$$

$$x^{(1)} = M^{-1}\begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

# Stationary Methods: Choosing the Splitting Matrix **M**

$$Ax = b$$

$$\boxed{\mathbf{x}^{(\mathbf{k}+\mathbf{1})} = \mathbf{x}^{(\mathbf{k})} + \mathbf{M}^{-1}\mathbf{r}^{(\mathbf{k})}} \iff \overbrace{M\left(x^{(k+1)} - x^{(k)}\right) = r^{(k)}}$$

- **M** should be easy to "invert" (we never actually "invert"; we solve $\mathbf{M}\left(\mathbf{x}^{(\mathbf{k}+\mathbf{1})} - \mathbf{x}^{(\mathbf{k})}\right) = \mathbf{r}^{(\mathbf{k})}$);
- from $\underbrace{\mathbf{x} = (\mathbf{I} - \mathbf{M}^{-1}\mathbf{A})\mathbf{x} + \mathbf{M}^{-1}\mathbf{b}}$, **M** should approximate **A**.

$\cancel{x} = \cancel{x} - M^{-1}Ax + M^{-1}b \iff M^{-1}Ax = M^{-1}b$

$Ax = b$

# Jacobi Method

$$D = \begin{bmatrix} a_{11} & 0 \\ 0 & a_{22} \end{bmatrix} \qquad D^{-1} = \begin{bmatrix} 1/a_{11} & 0 \\ 0 & 1/a_{22} \end{bmatrix}$$

$$\mathbf{x}^{(\mathbf{k}+\mathbf{1})} = \mathbf{x}^{(\mathbf{k})} + \underbrace{\mathbf{D}^{-\mathbf{1}}\mathbf{r}^{(\mathbf{k})}} \quad (*)$$

- $\mathbf{M} = \mathbf{D} = \mathrm{diag}\,(a_{11}, \ldots, a_{nn})$;
- $\mathbf{D}^{-\mathbf{1}}$ is trivial: reciprocals of the entries;
- the update is obtained by simply dividing $r_i^{(k)}$ by $a_{ii}$, that is, scaling the residual with the diagonal entry.

1) $x_i^{(k+1)} = x_i^{(k)} + \frac{1}{a_{ii}} r_i^{(k)}$ ← row $i$ of $(*)$

2) $\boxed{r = b - Ax}$

the ith row : $r_i^{(k)} = b_i - \sum_{j=1}^{n} a_{ij} x_j^{(k)}$

3) $$x_i^{(k+1)} = x_i^{(k)} + \frac{1}{a_{ii}} \left[ b_i - \underbrace{\sum_{j=1}^{n} a_{ij} x_j^{(k)}} \right]$$

$$= x_i^{(k)} + \frac{1}{\cancel{a_{ii}}} \left[ b_i - \cancel{a_{ii} x_i^{(k)}} - \sum_{j=1, j \neq i}^{n} a_{ij} x_j \right]$$

$$= \cancel{x_i^{(k)}} - \frac{\cancel{a_{ii} x_i^{(k)}}}{\cancel{a_{ii}}} + \frac{1}{a_{ii}} \left[ b_i - \sum_{j=1, j \neq i}^{n} a_{ij} x_j \right]$$

# Jacobi Method

If $a_{ii} \neq 0$, for $i = 1, \ldots n$, Jacobi Method is

$$x_i^{(k+1)} = \underbrace{\frac{1}{a_{ii}} \left[ b_i - \sum_{j=1, j \neq i}^{n} a_{ij} x_j^{(k)} \right]}, \quad ,$$

- $x_i^{(k)}$ is adjusted to mitigate the $i$-th component of the residual;
- each $x_i^{(k)}$ can be computed independently, so parallelization is easy;
- if $a_{ii} = 0$ and the system is non-singular, then reorder the equations;
- rearrange the equations to get the largest $a_{ii}$ in order to speed up the convergence of the algorithm.